ENGINEERING GUIDE · VERSION 2.0

Technical & Architecture Documentation

Architecture, data model, business rules, APIs, extension points, operations, and safe change procedures.

EGAS Employee Clearance System
Node.js · Express · PostgreSQL · Sequelize · React · Vite
16 August 2026

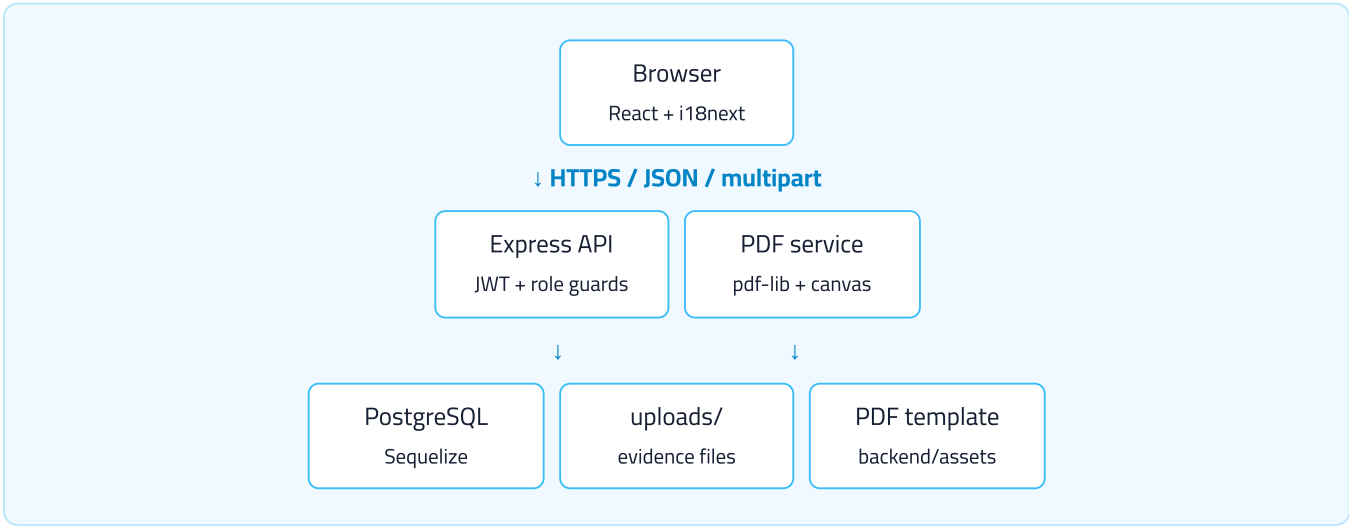
Audience and scope

This guide is for developers and database/operations staff who need to understand, maintain, extend, or troubleshoot the application. It is based on the live repository and running local instance—not a generic template.

> **Canonical sources:** schema in `backend/src/models/index.js`; departments in `backend/src/seed/departments.data.js`; authorization in the route files and auth middleware; current behavior in tests and running application.

1. System overview and architecture

The system digitizes EGAS employee clearance. A React single-page application calls an Express JSON API. Express authenticates JWTs, enforces role and department boundaries, stores relational state in PostgreSQL through Sequelize, stores uploaded evidence on disk, and generates a composited clearance PDF from an official template.



Technology choices

Layer	Technology	Responsibility
Frontend	React 18, Vite, React Router, Axios, Recharts	Role dashboards, forms, analytics, previews.
Localization	react-i18next	Arabic-first bilingual UI and RTL direction.
Backend	Node.js, Express 4	REST API, validation, business logic, file delivery.
Authentication	JWT, bcryptjs	Email/password login and password re-authentication.
Database	PostgreSQL, Sequelize	Users, templates, request snapshots, status/audit data.
Evidence/PDF	multer, pdf-lib, fontkit, canvas/image codecs	Upload validation, preview, template compositing.

> **Deployment shape:** development uses Vite on port 5173 and the API on 4000. When `frontend/dist` exists, Express serves the built SPA and API from one Node process.

2. Repository map and runtime

```
Egas-clearance-standalone/
├── backend/
│   ├── assets/                PDF template, Cairo fonts, demo evidence
│   ├── scripts/               smoke-test.js, stress-test.js
│   ├── src/
│   │   ├── config/db.js       connection + sequelize.sync({alter:true})
│   │   ├── middleware/        JWT and role/department guards
│   │   ├── models/index.js    all models and associations
│   │   ├── routes/            auth, departments, requests
│   │   ├── seed/              templates and demo data
│   │   ├── services/          request assembly + clearance PDF
│   │   └── server.js          application entry point
│   └── uploads/               runtime evidence; gitignored
├── frontend/
│   ├── public/fonts/          Cairo web fonts
│   └── src/
│       ├── api/client.js      Axios + JWT interceptor
│       ├── components/        reusable dashboard/UI pieces
│       ├── context/           auth and theme state
│       ├── locales/           en.json and ar.json
│       ├── pages/             role-level screens
│       └── App.jsx            route guards and routing
├── README.md                  operator/user overview
├── CLAUDE.md                  detailed engineering decisions
└── PROJECT_STATUS.md          chronological living status
```

Local commands

```
npm install
npm install --prefix backend
npm install --prefix frontend

# configure backend/.env, then:
npm run seed:dev --prefix backend
npm run dev

# production-like data (departments only; no demo users):
npm run seed:final --prefix backend

# compile frontend:
npm run build --prefix frontend
```

Required environment

```
PORT=4000
DATABASE_URL=postgres://USER:PASSWORD@HOST:5432/egas_clearance
JWT_SECRET=replace_with_a_long_random_secret
JWT_EXPIRES_IN=8h
```

Node.js 18+ and a reachable PostgreSQL instance are required. Never commit `backend/.env` or production credentials.

3. Relational data model

The application migrated from MongoDB to PostgreSQL while preserving the old nested request shape through `requestAssembly.js`. Natural keys are retained: user email is `users.userID`; department key is `departments.key`; requests use UUIDs.

Table	Purpose and important fields
departments	Live 13-department template: bilingual names, order, tier, oversight flag, signature mode.
department_checklist_items	Live template for IT's five itemized requirements.
users	Email PK, bcrypt hash, role, department assignment, optional IT item assignment, phone, forced-reset flag.
clearance_requests	Employee/departure facts, creator, overall status, File Management approval, access-removal audit fields.
request_departments	Per-request snapshot of every department plus signature, evidence metadata, signer identity, time, and oversight notes.
request_items	Per-request IT checklist snapshot plus item-level signature/evidence metadata.

Snapshot invariant

On creation, department names, order, tier, signature mode, and checklist items are copied into request-owned rows. Later template changes affect only future requests. Do not "simplify" reads by joining live department configuration into historical requests.

```
// RequestDepartment uniqueness
indexes: [{ unique: true, fields: ["requestId", "departmentKey"] }]

// RequestItem uniqueness
indexes: [{ unique: true, fields: ["requestDepartmentId", "key"] }]
```

> **Schema management:** startup currently calls `sequelize.sync({ alter: true })`. There is no migrations directory. This is convenient for one small deployment but is risky for controlled production changes; back up and test every schema edit before startup.

4. Business rules and authorization

Workflow state machine

- Departments 1–11 are Tier 1 and work concurrently.
- IT is itemized and completes only when all five request items are complete.
- Wages and Finance unlock only when every Tier-1 department completes.
- Reopening clears the affected signature/evidence and revokes prior File Management approval.
- All departments signed is insufficient for completion. `accessRevoked` is the final gate.

Role hierarchy

Role	Route/data boundary
file_management	All request summaries/evidence; create, reopen, approve, delete completed requests, PDF. Signer identity is redacted.
reviewer	Own department only, except Wages/Finance oversight. IT receives safe readiness flags.
admin	Manage only file_management and reviewer users; no request access.
super_admin	Manage only admin users; no request access. Exactly one account, created only through empty-database setup.

```
const MANAGEABLE_ROLES_BY_ACTOR = {
  admin: ["file_management", "reviewer"],
  super_admin: ["admin"],
};
```

Authorization is server-enforced. Frontend route guards improve UX but do not constitute security. JWT payloads identify the user, role, department, assigned IT item, oversight status, and forced-password-reset state.

> **Never weaken data redaction for convenience.** New endpoints must apply the same role, department, item, evidence, and signer-identity boundaries as existing request routes.

5. API and request lifecycle

All application routes are under `/api`. The frontend Axios client uses relative URLs and attaches `Authorization: Bearer <token>` from local storage.

Method/path	Purpose / principal authorization
GET /health	Process health check.
POST /auth/login	Public credential exchange.
GET /auth/setup-status POST /auth/setup	Public first-run check; setup creates the sole Super Admin only when user count is zero.
GET/POST /auth/accounts	Admin/Super Admin, filtered by manageable roles.
POST /auth/reset-password POST /auth/delete-account	Admin/Super Admin with role boundary and re-authentication.
POST /auth/set-new-password	Authenticated temporary-password replacement.
GET /departments	Public template data for setup/account forms.
GET /departments/coverage	Coverage gaps: missing admin, File Management, reviewers/items.
POST /requests	File Management creates request and snapshot rows.
GET /requests GET /requests/:id	Role-scoped listing/detail.
POST /requests/:id/sign	Standard reviewer sign-off plus evidence upload.
POST /requests/:id/items/:itemKey/sign	IT item sign-off plus evidence upload.
POST .../reopen	Authorized reviewer or File Management clears sign-off/evidence.
POST /requests/:id/approve	File Management evidence approval.
POST /requests/:id/revoke-access	IT final action; computes completed status.
GET /requests/:id/evidence/...	Permission-checked evidence streaming.
GET /requests/:id/pdf	Permission-checked generated clearance PDF.

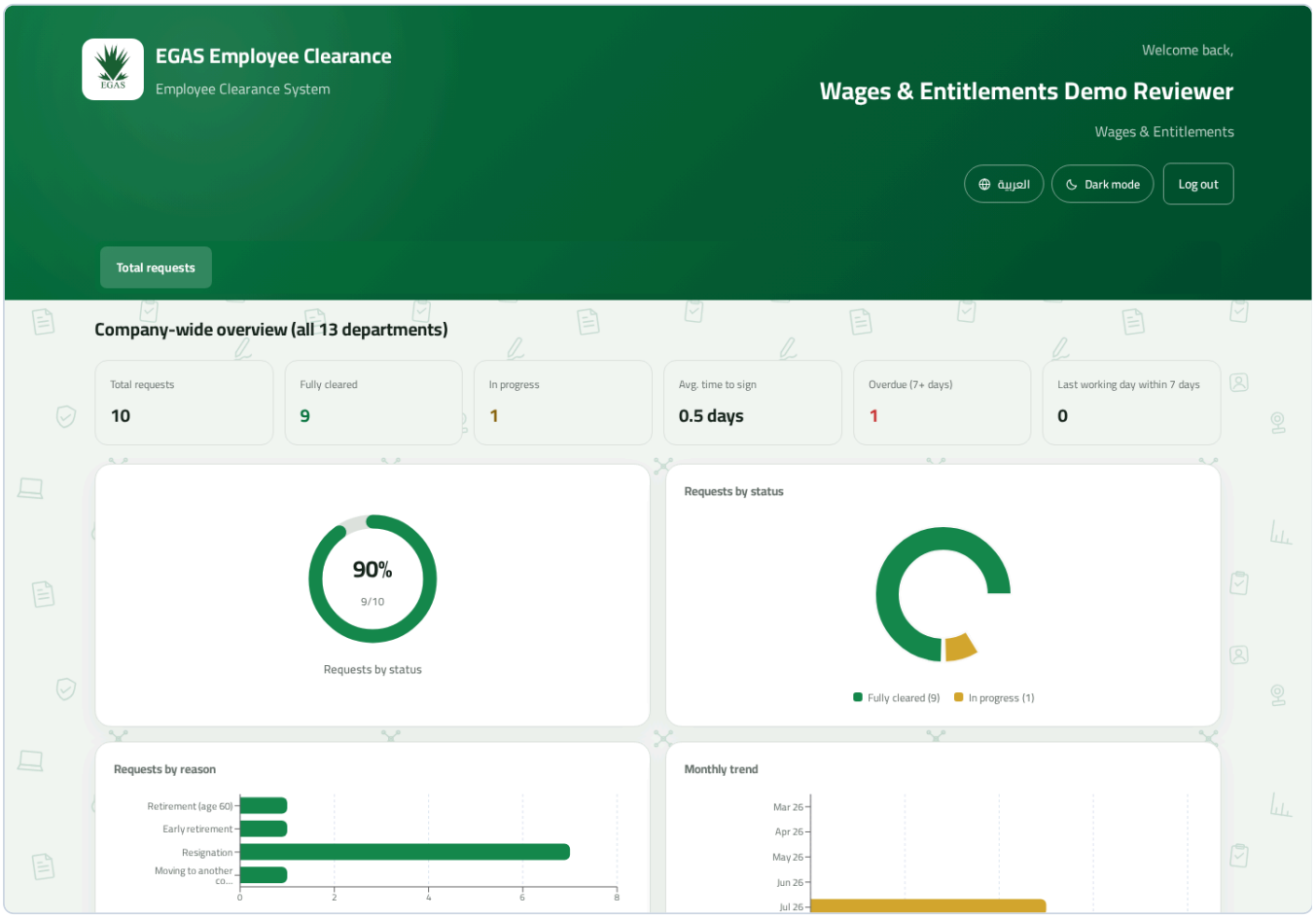
Write-route pattern

1. Authenticate and authorize role/department/item.
2. Fetch assembled plain request for business validation.
3. Re-authenticate current password for consequential actions.
4. Validate upload and transition prerequisites.
5. Update targeted Sequelize rows.
6. Re-fetch and call the shared status-sync logic.
7. Return only the role-appropriate projection.

6. Frontend architecture

`App.jsx` defines public routes and four protected dashboards. `RequireRole` redirects unauthenticated users, forces temporary-password replacement, and prevents wrong-role rendering.

```
/login              Login.jsx
/setup              FirstRunSetup.jsx
/set-new-password   SetNewPassword.jsx
/file-management    FileManagementDashboard.jsx
/reviewer           ReviewerDashboard.jsx
/admin              AdminDashboard.jsx
/super-admin        AdminDashboard.jsx (reused with different role scope)
```



The actual reviewer UI illustrates shared tables, charts, localization, and role-driven presentation.

State and reusable components

- `AuthContext.jsx`: login state, persisted token/user, logout.
- `ThemeContext.jsx`: light/dark theme.
- `DepartmentDashboard.jsx`: common reviewer layout and analytics.
- `SignaturePanel.jsx`: password + evidence interaction.
- `EvidencePreview.jsx`: permission-backed image/PDF preview.
- `RequestOversightGrid.jsx`: full 13-department oversight.
- `SetupCoverageWarning.jsx`: missing staffing/configuration warnings.

Localization rule

Every new user-facing string belongs in both `frontend/src/locales/en.json` and `ar.json`. Test English and Arabic, including RTL alignment, long labels, tables, modals, and charts. Avoid embedding English strings directly in components.

7. Evidence and clearance PDFs

Evidence is stored under `backend/uploads/<requestId>/`. The database stores URL, MIME type, and original name, while protected endpoints decide whether the current user can fetch the file.

Accepted uploads

Reviewer sign routes use multer disk storage and accept JPG, PNG, and PDF evidence. Validation must remain synchronized between frontend guidance, multer's filter/limits, preview support, seed data, and PDF compositing.

Generation pipeline

`backend/src/services/clearancePdf.js` loads the official form, embeds Cairo fonts for Arabic and Latin text, processes evidence images/PDFs, places signatures in department-specific coordinates, and returns the generated bytes. Do not replace the form without verifying dimensions and every placement coordinate.

> **Evidence lifecycle:** reopening or deleting a request involves both database state and files on disk. Maintain consistency on failures. Back up the database and upload directory together; one without the other is not a restorable clearance record.

Security considerations

- Do not expose `uploads/` as a public static directory.
- Preserve path normalization and authorization before file access.
- Do not trust original filenames for storage paths.
- Validate MIME/type and size server-side.
- Avoid logging passwords, JWTs, evidence bytes, or production database URLs.

8. Safe change recipes

Add or rename a department

1. Edit `backend/src/seed/departments.data.js` with stable key, bilingual names, unique order, tier, oversight flag, and signature mode.
2. Update the official PDF template and placement mapping if form rows change.
3. Run the appropriate department seed/upsert.
4. Create coverage accounts through Admin; for IT, ensure each item has one owner.
5. Test a newly created request and confirm old requests retained their snapshots.
6. Update translations, demos, smoke tests, README/engineering notes, and these manuals.

> **Do not change a department key casually.** It is a natural primary/foreign key and appears in users, snapshots, routes, demo accounts, and PDF mapping. Treat a key rename as a data migration.

Add a leaving reason

1. Add the stable enum value to `LEAVING_REASONS` in the model.
2. Add English/Arabic labels and any formatting helper mapping.
3. Exercise request creation, list display, analytics grouping, PDF output, and seed/test fixtures.
4. Back up before allowing Sequelize alter to modify the production enum.

Add a database field

1. Decide whether it belongs to live configuration, the request snapshot, or both.
2. Add model validation and explicit API input validation.
3. Update assembly/projection/redaction logic so it cannot leak to unauthorized roles.
4. Update create/write routes, frontend form/display, both locales, PDF output if applicable, tests, and documentation.
5. Test schema alteration on a production-like copy before deployment.

Add a role or permission

Update the PostgreSQL enum, login token payload, middleware, route guards, data projections, dashboard routing, account-management hierarchy, coverage checks, translations, tests, and documentation. Begin with server-side authorization; UI visibility comes afterward.

Changing or removing data safely

Preferred paths

Need	Preferred method	Avoid
Create/reset/delete a user	Admin/Super Admin UI or authenticated API.	Direct SQL that bypasses hashing, role rules, or audit intent.
Change department templates	Edit source-of-truth data and run the idempotent upsert.	Editing historical request snapshot rows globally.
Delete a clearance request	Authorized File Management action on a completed request, which handles DB and upload directory.	Deleting only the database row or only upload files.
Correct evidence	Reopen the department/item, then re-sign.	Replacing a file directly on disk.
Correct historical production data	Approved, backed-up, transaction-wrapped migration with verification.	Ad hoc SQL in a live shell.

Transaction pattern for a one-off migration

```
BEGIN;
-- Identify exact rows first; preserve a query/result record.
SELECT ... FOR UPDATE;

-- Apply the smallest explicit update.
UPDATE ... WHERE ...;

-- Verify row count and invariants before commit.
SELECT ...;
COMMIT;  -- or ROLLBACK on any uncertainty
```

Use a maintenance window for material changes. Take a tested backup, record affected primary keys and expected counts, and keep rollback steps. Never edit `passwordHash` manually; use application reset flows.

9. Deployment, testing, and recovery

Release checklist

- ☐ Review `git diff`; exclude secrets, uploads, temporary screenshots, and unrelated changes.
- ☐ Back up PostgreSQL and `backend/uploads` together.
- ☐ Install dependencies from lockfiles with a supported Node version.
- ☐ Build the frontend successfully.
- ☐ Apply/verify schema change on a staging or restored copy.
- ☐ Run the smoke test against a seeded non-production database.
- ☐ Verify Arabic and English role dashboards.
- ☐ Verify login, sign, reopen, approve, evidence preview, PDF, and access removal.
- ☐ Deploy and check `GET /api/health`.
- ☐ Confirm required coverage (Admin, File Management, departments, IT items).

Testing commands

```
# terminal 1
npm run dev

# terminal 2 — uses a running, seeded server
node backend/scripts/smoke-test.js

# frontend compilation
npm run build --prefix frontend

# optional load exercise; review the script before running
node backend/scripts/stress-test.js
```

The smoke test creates throwaway accounts and requests and exercises role boundaries and the end-to-end transition. Never aim destructive or load-oriented scripts at production without explicit review.

Backup unit and restore test

A complete backup consists of the PostgreSQL database, upload directory, deployed code revision, official PDF template, and environment/configuration kept in an approved secrets system. Restore into an isolated environment and verify evidence preview and PDF generation—not just row counts.

10. Risks and maintainer checklist

Risk	Why it matters	Mitigation
Sole Super Admin lockout	No other role can reset it.	Secure credential escrow and tested organizational recovery procedure.
Automatic schema alteration	`sync({alter:true})` can make production changes at startup.	Backups, staging rehearsal, controlled Node/Sequelize versions; consider formal migrations.
Filesystem evidence	DB-only backup is incomplete; horizontal scaling needs shared storage.	Joint backups; persistent protected volume; consider object storage if scaling.
JWT in localStorage	XSS would expose session tokens.	Strong CSP/input hygiene; avoid unsafe HTML; consider secure HttpOnly cookies in a future security hardening project.
Public contact/setup endpoints	Intentional unauthenticated surface.	Keep returned fields minimal; setup must re-check zero users atomically.
Documentation drift	Fast-changing rules can make procedures unsafe.	Update README, CLAUDE.md, PROJECT_STATUS, tests, and manuals in the same change.

Definition of done for a change

- ☐ Business invariant and authorization impact are explicit.
- ☐ Backend validation and data redaction are tested.
- ☐ Existing in-flight request snapshots remain valid.
- ☐ Both languages and RTL are correct.
- ☐ Evidence/PDF implications are verified.
- ☐ Build and smoke test pass.
- ☐ Deployment and rollback steps exist.
- ☐ Documentation is updated.

> **Core architectural principle:** preserve historical request snapshots and enforce every permission on the server. UI controls, documentation, and database discipline support those two invariants; they do not replace them.

Verified against repository and local application state on 16 August 2026. Review the latest source and project status before applying production changes.